Princess Sumaya University for Technology

King Hussein School of Computing Sciences

Computer Science Department

Operating Systems Project

Performance Evaluation of CPU Scheduling Algorithms

Students:

Ghaith Zakaria	GHA20230856@std.psut.edu.jo	- Shortest Job First
Asaad Ghanma	ASA20230648@std.psut.edu.jo	- First Come First Serve
Abdulla Dahouk	ABD20230901@std.psut.edu.jo	- Round Robin
Khaled Thabet	KHA20220749@std.psut.edu.jo	- Shortest Remaining Time First
Hammam Al Rawabdeh	HAM20230438@std.psut.edu.jo	- Priority Scheduling (Non-preemptive)

Table of Contents

1. Introduction
2. CPU scheduling background
3. Methodology
    a. Tools used & overview
    b. Data sample
    c. Algorithms
4. Results
5. Conclusion
6. References

1. Introduction

CPU scheduling is one of the most crucial parts of an operating system, this is because the CPU scheduler chooses which process enters the CPU first, and the order of how processes are picked to be executed inside the CPU. There are multiple CPU scheduling algorithms, and we will be discussing some of them thoroughly and showcasing their advantages and disadvantages, more specifically:

- First come first serve algorithm
- Round robin
- Shortest job first
- Shortest remaining time first
- Priority scheduling (Non-preemptive)

The purpose of this project was to draw comparisons between all the algorithms according to their average waiting time and their average turnaround time, regarding the data sample chosen, which is mentioned in the methodology section.

2. Background on CPU scheduling

As previously mentioned, CPU scheduling is important because it decides which process in the ready queue gets executed by the CPU. There are 2 different types of CPU scheduling, preemptive and non-preemptive:

- Non-preemptive scheduling means that once a CPU is assigned to a process it does not leave it unless it terminates or gets interrupted for waiting for I/O operations
- Preemptive scheduling means that once a CPU is assigned to a process, the process could be forced out of CPU by the scheduler

(More about the CPU scheduling algorithms is mentioned in the algorithms section in the methodology)

3. Methodology

In this project, multiple tools and references were used to make working on this project as a team easier and more efficient. For starters, Hammam Alrawabdeh created a GitHub repository for the project, including a README section and all the files related to the project. Then each of the members in the group worked on their own algorithm on C++ and uploaded it onto the repository. We then tested out the project multiple times and made sure that it runs as smoothly and as efficiently as it could, we then chose a test sample made up of 5 processes and tested this sample using all of the algorithms, the code automatically calculated the turnaround time and the waiting time, thus giving us the ability to draw conclusions accordingly.

a. Tools used and overview:

As previously stated, we used a GitHub repository to have all of our files in one place and make sure that our work looks presentable and understandable for users. We used C++ to code our project since C++ is an object oriented language which allowed us to create classes easily. A class for processes was created (Process.cpp), which included the process's ID, arrival time, priority, burst time, completion time, turnaround time, waiting time, remaining time, and a boolean variable isCompleted, with a default constructor that would automatically set all values to 0 and the ID to "" and isCompleted to false . The user only has to enter the arrival time, burst time, and priority (the lower the number, the more priority the process has), the user then enters which algorithm they would like to test out, and the average turnaround and waiting time is calculated and displayed.

b. Data sample

For the data sample, we decided to use the following example, it is a fair and thorough example that could be applied to all the algorithms. This set of processes is a good enough set to help us draw our conclusions about each algorithm. **(Conclusions drawn are strictly relative to this set of data samples)**

PID	Arrival Time	Burst Time	Priority
P1	0	6	3
P2	1	2	1
P3	2	2	4
P4	5	4	2
P5	7	1	5

Quantum = 3

c. Algorithms

Each CPU scheduling algorithm has its own advantages and disadvantages, meaning that there is no universally best CPU scheduling algorithm, since some algorithms work better than others for specific data samples.

- **First come first serve algorithm (FCFS)** is relatively straightforward, whichever process first enters the ready queue gets executed first. This might seem like a very minimalistic approach, but in reality it has its own advantages, some of which are:
    - No chance of process starvation since no specific process is prioritized over another
    - It's easy to implement and it's a simple CPU scheduling algorithm

  On the other hand, it has a lot of disadvantages that make it one of the slower algorithms for CPU scheduling, some of which include:
    - It is a non-preemptive algorithm, meaning that some processes will have to wait a long time to get executed if processes ahead of it are large and time consuming
    - The average turnaround time for FCFS is usually more than the average turnaround time of other algorithms

- **Priority Scheduling (Non preemptive)** is an improvement of older CPU scheduling algorithms, like FCFS and Round Robin with higher priority to the more important processes. "Each process gets a priority, and then the runnable process with highest priority gets to run", reads Andrew S. Tanenbaum. Non-Preemptive Priority Scheduling processes keep running without interruption until they finish. For this project higher priority numbers indicate higher priority levels, and when two processes arrive at the same time, the process that comes earlier in the file will be executed first.

This algorithm is beneficial on systems where quick execution for critical tasks, including real-time systems and banking is required. Its advantages include:

- simple implementation
- lower context-switching overhead

while its main disadvantage is:

- starvation, where low-priority processes may wait indefinitely.

Priority based scheduling concepts and their evolved variants such as Round Robin and Multilevel Feedback Queues (MLFQ) are still in use with modern operating systems.

- **The Shortest Job First (SJF)** algorithm is a non-preemptive scheduling algorithm , it works by picking the process with the smallest burst time (execution time) and with the earliest arrival time out of the available processes in the waiting queue first, hence the name .By definition it is a greedy algorithm. This algorithm is considered to be better than the First Come First Serve (FCFS) algorithm because it allows smaller processes to be completed before larger processes , which usually reduces the average response time for all processes.Usually , the Shortest Job First algorithm is used for long term scheduling and for the jobs running in batches , where job burst times are already known.To implement this algorithm , you need to know or approximate the burst time of each process ; which usually limits its usage to environments that have accurate estimates of process burst times or the usage of estimation methods such as weighted average of previous execution times for the process .Moreover, the Shortest Job First algorithm may cause process starvation; Since a long process may wait indefinitely to be completed while short processes keep flowing into the waiting process queue taking its turn to be executed.

- **Round Robin (RR)** algorithm is the most used Scheduling algorithm of all time.

  It is a preemptive Scheduling algorithm where an amount of time

  (called time quantum) specifies the maximum amount of time a process can spend in the Central Processing Unit (CPU) at a time , once the time quantum of a process ends the process is preempted out of the (CPU) and sent back to the waiting queue which is implemented as a First In First Out (FIFO) queue.

  The reason for Round Robin algorithm wide usage is that it does not depend on the processes burst time - which is often hard to predict - but only on the time quantum specified so it can be truly implemented in the operating system without any additional components or any knowledge about the processes the system uses.Moreover, Round Robin algorithm gives each process an equal amount of CPU time; which prevents process starvation. However, Round Robin algorithm has two main disadvantages, the first is the CPU context switch added overhead; since it is a preemptive algorithm. The second is the huge impact of the time quantum selection on the algorithm; a too small time quantum in respect to process context switch time in the system causes the overhead of context switches to waste system resources and reduce the throughput of the system , on the other side a too big time quantum makes this algorithm behave like the First Come First Serve Algorithm (FCFS) , losing all the advantages of applying Round Robin algorithm in the first place .

- **Shortest Remaining Time First (SRTF)** is a preemptive version of SJF (Shortest Job First) scheduling to improve average waiting time over FCFS and Priority Scheduling (which are non-preemptive). Abraham Silberschatz (In Operating System Concepts) claims that the SRTF algorithm is the preemptive version of SJF, and “When a new process enters the ready queue, its burst time is compared with the remaining time of the process currently being executed, and if it is less, the process being executed is preempted.” In SRTF, the process with the minimum remaining process burst-time is always executed by the CPU. Each time a new process is added to the ready queue, the scheduler examines the remaining time of the process that is currently running to determine whether the new process's burst time is shorter. If so, the running process goes back to the ready queue and the new process begins to run. In the case of two process with equal time to be removed from the queue, the process that entered the queue earlier is served first by FCFS principle of tie breaking.

  SRTF is useful in systems where it is important to minimize average waiting time. For instance, in transaction processing systems or in databases, in which short queries are handled first, the two are handled dynamically, thus improving the throughput. SRTF has an important advantage: it has a minimal average waiting time of any of the preemptive scheduling algorithms. Another advantage is that jobs that are short do not wait for long running jobs to finish, so that interactive or light weight processes have less response time. There are drawbacks, however. The biggest problem is that of starvation, a long process that can be delayed or put on hold as long as the shorter jobs come in. The other disadvantage is that burst times in practice will be unknown and the algorithm relies on the knowledge of future burst times, which is not possible in real systems. Furthermore, it could result in high context switching overhead due to preemption at each arrival. A number of operating systems have methods to solve the burst time uncertainty problem such as exponential averaging for predicting the next burst from previous bursts. SRTF concepts continue to influence modern systems, but are generally not implemented as they are in the original SRTF; in modern systems, such as Linux and Windows, scheduling methods exist that are similar to SRTF's but try to balance optimality and practicality.

4. Results

After running the processes example through our algorithms we got the following results:

FCFS

P1 (0-6)	P2 (6-8)	P3 (8-10)	P4 (10-14)	P5 (14-15)

Average Waiting Time: 4.60

Average Turnaround Time: 7.60

Priority Scheduling (Non-preemptive)

P1 (0-6)	P2 (6-8)	P4 (8-12)	P3 (12-14)	P5 (14-15)

Average Waiting Time: 5.00

Average Turnaround Time: 8.00

Shortest Job First

P1 (0-6)	P2 (6-8)	P5 (8-9)	P3 (9-11)	P4 (11-15)

Average Waiting Time: 3.80

Average Turnaround Time: 6.80

Round Robin

P1 (0-3)	P2 (3-5)	P3 (5-7)	P4 (7-10)	P5 (10-11)	P1 (11-14)	P4 (14-15)

Average Waiting Time: 4.20

Average Turnaround Time: 7.20

Shortest Remaining Time First

P1 (0 - 1)	P2 (1 - 3)	P3 (3 - 5)	P4 (5 - 7)	P5 (7 - 8)	P4 (8 - 10)	P1 (10 - 15)

Average Waiting Time: 2.20

Average Turnaround Time: 5.20

5. Conclusion

In conclusion, this project helped us evaluate how the CPU performs its scheduling with the 5 different algorithms, each algorithm was implemented on C++ and tested using the same sample set of processes for us to be able to calculate the average turnaround and waiting time for each of them.

We can see that based on the results, SRTF (shortest remaining time first) performed best for our example data set, with the lowest average turnaround time and the lowest average waiting time, showing that preemptiveness could significantly improve the efficiency of CPU scheduling. We can also see that SJF (shortest job first) performed pretty well too, since it prioritized shortest jobs first and our processes' burst times were also relatively short. Round Robin gave each process equal CPU time, thus providing a bit of a balanced approach to CPU scheduling. Finally, both FCFS (first come first serve) and Priority scheduling had the highest average turnaround and waiting times, this is because they do not prioritize shorter burst times over other burst times, which is a worse approach for this data sample only.

Meaning that SJF and SRTF perform better when the process set contains shorter burst times, especially when short processes are mixed with longer ones. While priority scheduling has its own scenarios where it is better and more efficient. And Round Robin is always a more balanced approach if there are no specific priorities set for the processes. Moreover, even FCFS could be a very efficient approach, especially in more simple, direct scenarios. **It's all relative to the system's goals.**

To sum things up, this project helped demonstrate how CPU scheduling is crucial when it comes to system performance. It also proved that there is no single best algorithm for CPU scheduling, and that the operating system should always look for the most suitable algorithm when choosing or designing scheduling algorithms.

6. References

- *Operating Systems slides - PSUT Elearning*
- *"FCFS - First Come First Serve CPU Scheduling." GeeksforGeeks, GeeksforGeeks, 6 Jan. 2026, www.geeksforgeeks.org/dsa/first-come-first-serve-cpu-scheduling-non-preemptive/.*
- *Andrew S. Tanenbaum & Herbert Bos, Modern Operating Systems, 4th Edition, Pearson.*
- *Abraham Silberschatz, Peter B. Galvin, Greg Gagne, Operating System Concepts, 10th Edition.*
- *"Program for Shortest Job First (or SJF) CPU Scheduling: Set 1 (Non- Preemptive)." GeeksforGeeks, GeeksforGeeks, 23 July 2025, www.geeksforgeeks.org/dsa/program-for-shortest-job-first-or-sjf-cpu-scheduling-set-1-non-preemptive/.*
- *"Round Robin Scheduling - Tpoint Tech." Www.Tpointtech.Com, Tpoint Tech, 7 Jan. 2026, www.tpointtech.com/os-round-robin-scheduling-algorithm.*
- *Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley. (Chapter 5 – CPU Scheduling)*
- *Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4th ed.). Pearson. (Section 2.4)*
- *Stallings, W. (2017). Operating Systems: Internals and Design Principles (9th ed.). Pearson. (Chapter 9)*
- *MIT 6.828 Operating System Engineering Course Materials: Scheduling Lectures*
- **For gantt charts:** *CPU Scheduling Calculator, aadhil2k4.github.io/Process_Scheduling_Calculator/#/. Accessed 15 May 2026.*